Exercice guidé — Simuler la loi des grands nombres en Python

On lance une pièce équilibrée. On note **Pile** lorsque le nombre tiré au hasard est inférieur à 0,5, et **Face** sinon. Ainsi, chaque issue a une probabilité de $\frac{1}{2}$.

La fonction Python ‘random()‘ (importée depuis ‘random‘) renvoie un nombre compris entre 0 inclus et 1 exclu. Le programme ci-dessous simule n lancers et mémorise, après chaque lancer, la fréquence de Pile observée jusque-là.

À faire : compléter les cinq blancs. Pour le premier, réutiliser l’idée du cours : comparer ‘random()‘ à 0,5.

```
from random import random

def simuler(n):
    # Le compteur commence a zero : aucun Pile n'est sorti.
    nb_pile = 0
    frequences = []  # on y rangera les frequences successives

    # i prend les valeurs 1, 2, ..., n (nombre de lancers deja faits).
    for i in range(1, n + 1):
        if ______________________________:  # un Pile ?
            ______________________________  # actualiser le compteur

        f = ______________________________  # frequence de Pile apres i lancers
        ______________________________      # ajouter f a la liste

    return ______________________________  # renvoyer toutes les frequences
```

1. Pour ‘n = 1‘, quelles sont les deux valeurs possibles de la première fréquence ?
2. À la ligne ‘f = ...‘, pourquoi divise-t-on par ‘i‘ et non par ‘n‘ ?
3. Compléter : après i lancers, la fréquence de Pile est

$$f_i = \frac{\text{nombre de Pile obtenus après } i \text{ lancers}}{\qquad\qquad}.$$

Le programme suivant utilise la fonction précédente. La liste ‘frequences‘ contient la fréquence de Pile après 1 lancer, puis après 2 lancers, et ainsi de suite jusqu’à 1 000 lancers. Compléter les éléments manquants pour afficher la courbe. (Le module ‘matplotlib‘ doit être disponible dans l’environnement Python.)

```
import matplotlib.pyplot as plt

n = 1000
frequences = simuler(n)

# L'axe horizontal indique le nombre de lancers.
plt.plot(______________________________, frequences, label="Frequence de Pile")
# Cette droite horizontale represente la probabilite theorique de Pile.
plt.axhline(_________________, color="red", linestyle="--", label="Probabilite : 0,5")
plt.xlabel("Nombre de lancers")
plt.ylabel("Frequence observee")
plt.ylim(0, 1)
plt.legend()
______________________________  # afficher le graphique
```

Aide : ‘range(1, n + 1)‘ fournit les entiers de 1 à n. La commande ‘plt.show()‘ affiche le graphique.

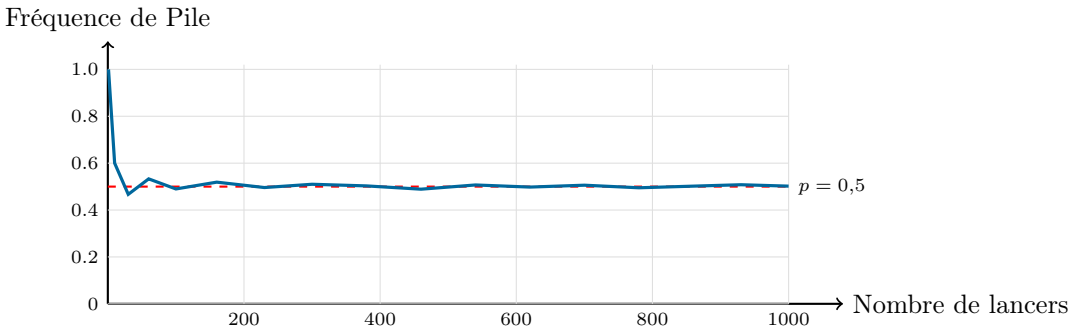
On veut comparer, cette fois, les fréquences finales des deux issues après les n lancers. Dans un diagramme en bâtons, chaque bâton représente une issue et sa hauteur représente sa fréquence observée. Compléter le programme ci-dessous à l'aide de 'frequences[-1]', qui désigne le dernier élément de la liste.

```
# A la fin, la frequence de Face complete celle de Pile a 1.
f_pile = frequences[-1]
issues = ["Pile", "Face"]
frequences_finales = [f_pile, ___________________]

# bar() trace un baton pour chaque issue.
plt.figure()  # ouvrir une nouvelle figure
plt.bar(issues, frequences_finales, color=["royalblue", "orange"])
plt.axhline(y=________, color="red", linestyle="--", label="Probabilite : 0,5")
plt.ylim(0, 1)
plt.ylabel("Frequence observee")
plt.legend()
plt.show()
```

Que représentent les hauteurs des deux bâtons ? Compare-les à la ligne rouge, puis refais la simulation plusieurs fois.

Voici un exemple de courbe obtenue lors d'une simulation de 1 000 lancers. Elle est donnée à titre illustratif : une nouvelle exécution produira généralement une autre courbe.



1. Au début, la fréquence varie beaucoup. Vers la fin, semble-t-elle encore varier ? Décrire ce que montre la courbe.

   _____

2. À 1 000 lancers, la fréquence de cet exemple vaut environ _____. Est-elle exactement égale à la probabilité 0,5 ?

3. La fréquence se rapproche-t-elle de 0,5 à chaque lancer, sans jamais s'en éloigner ? Justifier à l'aide de la courbe.

   _____

4. Exécuter le programme plusieurs fois. Les courbes sont-elles identiques ? La fréquence finale est-elle toujours la même ?

   _____

Modifier la valeur de 'n' et relever la fréquence finale obtenue pour chaque nombre de lancers.

Nombre de lancers	10	100	1 000	10 000
Fréquence finale	_____	_____	_____	_____

Les fréquences obtenues sont-elles toujours de plus en plus proches de 0,5 ?

_____

Corrigé — Simulation de la loi des grands nombres

1. Fonction de simulation

Voici la fonction complétée. À chaque tour de boucle, ‘i‘ est le nombre de lancers déjà effectués.

```
from random import random

def simuler(n):
    # Le compteur commence a zero : aucun Pile n'est sorti.
    nb_pile = 0
    frequences = []

    # i prend les valeurs 1, 2, ..., n.
    for i in range(1, n + 1):
        if random() < 0.5:
            nb_pile += 1

        f = nb_pile / i
        frequences.append(f)

    return frequences
```

Explications des blancs : on compare le nombre aléatoire à 0,5 ; si le test est vrai, on ajoute 1 au compteur de Pile. La fréquence après i lancers est `nb_pile / i` ; on la range dans la liste avec `append`, puis on renvoie la liste complète.

1. Pour $n = 1$, la fréquence vaut 1 si le premier lancer donne Pile, et 0 s'il donne Face.
2. On divise par i car seuls i lancers ont été effectués à cette étape. Le nombre n est le nombre total de lancers prévu.
3. La formule est $f_i = \dfrac{\text{nombre de Pile après } i \text{ lancers}}{i}$.

2. Courbe des fréquences successives

Le premier programme de tracé complété est :

```
import matplotlib.pyplot as plt

n = 1000
frequences = simuler(n)

# Abscisses : 1, 2, ..., n lancers.
plt.plot(range(1, n + 1), frequences, label="Frequence de Pile")
# La droite indique la probabilite theorique de Pile.
plt.axhline(0.5, color="red", linestyle="--", label="Probabilite : 0,5")
plt.xlabel("Nombre de lancers")
plt.ylabel("Frequence observee")
plt.ylim(0, 1)
plt.legend()
plt.show()
```

‘range(1, n + 1)‘ construit les abscisses correspondant aux fréquences. La ligne horizontale doit être placée à $y = 0{,}5$, qui est la probabilité théorique.

3. Diagramme en bâtons

La fréquence de Face est le complément de celle de Pile : les deux fréquences ont pour somme 1. Le programme complété est :

```
# La derniere valeur de la liste est la frequence finale de Pile.
f_pile = frequences[-1]
issues = ["Pile", "Face"]
frequences_finales = [f_pile, 1 - f_pile]

# Un nouveau graphique, avec un baton par issue.
plt.figure()
plt.bar(issues, frequences_finales, color=["royalblue", "orange"])
plt.axhline(y=0.5, color="red", linestyle="--", label="Probabilite : 0,5")
plt.ylim(0, 1)
plt.ylabel("Frequence observee")
plt.legend()
plt.show()
```

Le premier bâton donne la fréquence de Pile ; le second, celle de Face. La ligne rouge marque la probabilité théorique 0,5.

Résultats obtenus avec Python

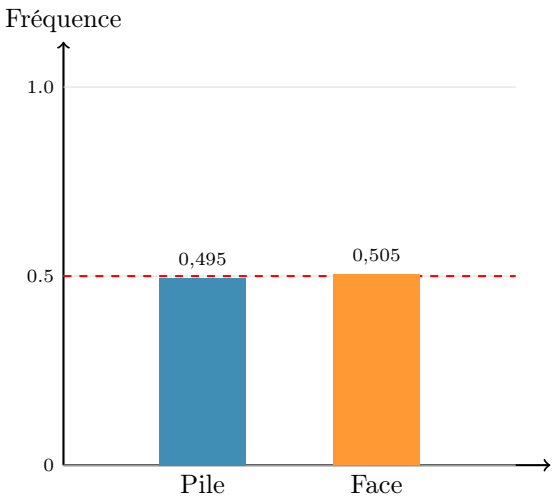
Pour pouvoir reproduire les nombres ci-dessous, on fixe la graine du générateur à 2026, puis on simule 10 000 lancers. Les fréquences aux étapes intermédiaires sont lues dans la même simulation.

```
from random import seed
seed(2026)
frequences = simuler(10000)
for n in (10, 100, 1000, 10000):
    f = frequences[n - 1]
    print(n, round(f, 4), round(1 - f, 4))
```

La sortie est :

```
Nombre de lancers | Frequence Pile | Frequence Face
               10 |         0.3000 |          0.7000
              100 |         0.4300 |          0.5700
             1000 |         0.4950 |          0.5050
            10000 |         0.5122 |          0.4878
```

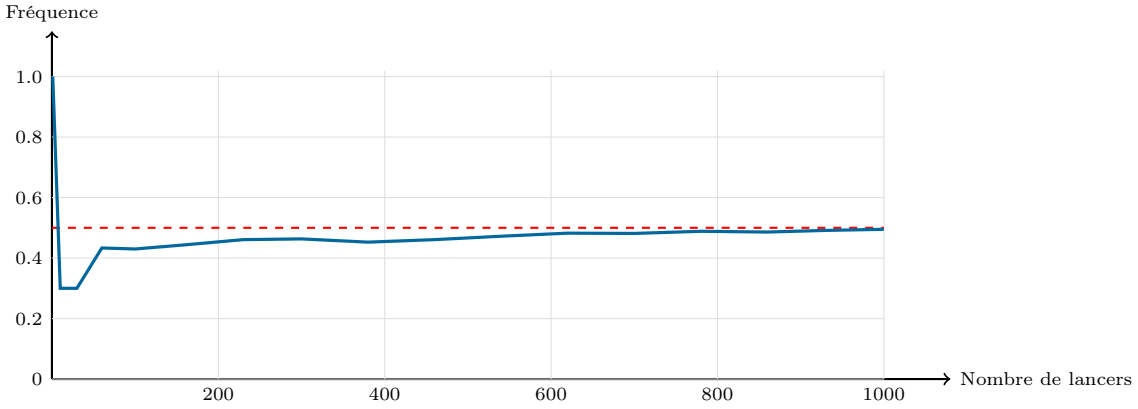
Dans cette même simulation, après les 1 000 premiers lancers, on obtient 495 Pile et 505 Face. Le diagramme suivant représente ces fréquences finales (0,495 et 0,505) :



Les deux bâtons sont presque de la même hauteur et proches de la ligne théorique rouge. Ils ne sont pas exactement à 0,5, car la simulation est aléatoire.

Interprétation et conclusion

Trajectoire obtenue avec Python (graine 2026), pour les 1 000 premiers lancers :



1. Sur la courbe illustrative de la fiche, la fréquence à 1 000 lancers est environ 0,502. Dans le tracé Python avec la graine 2026, elle vaut 0,495. Ces deux fréquences sont proches de 0,5, sans lui être égales.
2. La fréquence ne se rapproche pas de 0,5 à chaque lancer : elle fluctue. On le voit notamment au début de la courbe.
3. Les exécutions ne donnent pas toutes les mêmes courbes ni la même fréquence finale. La graine ‘2026‘ est fixée uniquement pour rendre l’exemple numérique reproductible.
4. Ici, l’écart à 0,5 vaut 0,005 à 1 000 lancers et 0,0122 à 10 000 lancers. Cet exemple rappelle que le rapprochement n’est pas monotone ; la loi des grands nombres décrit une tendance générale.